

Módulo 05 — Sistemas multiagênticos e MCP

Cada slide da apresentação explicado em linguagem simples: termos, o que falar e o que responder.

1. Slide a slide

Slide a slide

Este guia explica cada slide da apresentação em linguagem simples: o que significa cada termo, o que falar e o que responder se perguntarem. Leia uma vez inteiro antes da aula; durante a aula, as notas do PPTX bastam.

Se só der tempo de entender uma coisa: multiagente é dividir um trabalho entre vários agentes com papéis diferentes; só compensa quando as partes são independentes. **MCP** é um padrão para "plugar" ferramentas e dados em qualquer aplicação com IA, como uma tomada universal: quem faz o servidor não precisa saber quem vai usá-lo.

Antes da aula (uns 15 minutos):

1. `pnpm -F @mentoria/curador m05:edicao:solucao` (vários minutos): gera a edição e o relatório do time.
2. `pnpm -F @mentoria/ex05-mcp demo:protocolo` (instantâneo, não usa modelo).
3. `pnpm -F @mentoria/ex05-mcp demo:time` (~2 min) uma vez, para ver os papéis falando.

Slide 1 — Capa

Diga: *"Na aula passada tivemos um agente. Hoje, um time de agentes, e um jeito padrão de eles acessarem o mundo."*

Slide 2 — Hoje

Os objetivos. Não leia a lista.

Slide 3 — A frase da aula

Divida em vários agentes só quando as partes forem independentes.

Independentes: uma parte não precisa esperar nem saber o que a outra decidiu. Exemplo: pesquisar três assuntos diferentes ao mesmo tempo.

Slide 4 — Divisória "1 · Padrões multiagente"

"Padrão" aqui é uma forma conhecida de organizar o time.

Slide 5 — Cinco formas de organizar um time

Use a **analogia da cozinha de restaurante**:

- **Pipeline** (linha de montagem): cada cozinheiro faz uma parte e passa o prato adiante. No curador: coletor → classificador → redator → revisor.
 - **Supervisor**: o **chef** decide, a cada pedido, quem faz o quê. Um agente central escolhe qual "trabalhador" chamar.
 - **Hierárquico**: numa cozinha grande, o chef comanda **chefs de praça**, e cada um comanda a sua equipe. Supervisores de supervisores.
 - **Handoff** (passar a vez): o garçom recebe um pedido de sobremesa e **passa** para o confeitiro. Comum em atendimento: triagem → financeiro → suporte técnico.
 - **Gerador + revisor**: o cozinheiro faz, o chef **prova** e devolve com um comentário, até ficar bom. No curador: redator ⇔ revisor.
-

Slide 6 — O time do curador

O desenho mostra o time do projeto:

- **Coletor**: um **agente** de verdade (Aula 4): decide quais fontes ler.
- **Classificador**: a skill e os guardrails das Aulas 3 e 4. É uma decisão estruturada, não um agente.
- **Redator**: escreve o resumo.
- **Revisor**: confere o resumo, primeiro **com código** (tamanho, número de frases) e depois com um juiz LLM (fidelidade, Aula 3). Se reprovar, o redator tenta de novo com o motivo.
- **MCP fontes / MCP edição**: os dois "servidores" de onde vêm as notícias e para onde vai a edição (explicados no bloco 3).

Ponto importante: quem liga os papéis é o **código** (um pipeline fixo). Ligue com a Aula 4: só é agente onde há escolha de passos.

Slide 7 — Divisória "2 · Quando é exagero"

Passe direto.

Slide 8 — Duas visões, um dia de diferença

Dois textos muito citados, publicados em junho de 2025:

- **Anthropic:** o recurso de pesquisa deles usa um agente-líder que dispara vários subagentes **em paralelo** (cada um pesquisa uma parte). Nas avaliações internas, foi **90% melhor** que um agente sozinho. Mas gastou cerca de **15 vezes mais tokens** que um chat comum. Eles mesmos dizem que não serve para tarefas em que todos precisam do mesmo contexto, como programação.
- **Cognition** (empresa de agentes de programação): "não construa multiagentes". Agentes trabalhando em paralelo, sem ver o que o outro decidiu, tomam **decisões que se contradizem**. O exemplo deles: um agente desenha o fundo de um jogo num estilo e outro desenha o personagem em outro estilo, e o resultado não combina.

O ponto em comum (diga): multiagente compensa quando as partes são independentes e podem rodar ao mesmo tempo. Quando dependem umas das outras, dividir cria conflito e custo.

Perguntas para fazer antes de dividir:

1. As partes são independentes?
2. Cada papel precisa de informações, ferramentas ou permissões diferentes? (Separar quem só lê de quem publica é uma boa razão.)
3. O ganho paga o custo extra?
4. Um pipeline simples em código não resolve?

Slide 9 — Divisória "3 · MCP"

MCP = *Model Context Protocol* ("protocolo de contexto para modelos"). **Protocolo:** um conjunto de regras para dois programas conversarem.

Slide 10 — Uma tomada universal

O problema: cada aplicação com IA precisa se conectar a várias fontes (arquivos, bancos de dados, sistemas). Sem um padrão, cada combinação vira um conector diferente.

A solução: um padrão de "tomada". Quem cria um servidor MCP (o "aparelho") não precisa saber em que aplicação (a "casa") ele vai ser ligado.

Os papéis:

- **Host:** a aplicação com IA (um chat, um editor de código, o nosso time).

- **Cliente:** o "conector" dentro do host; um para cada servidor.
- **Servidor:** quem oferece ferramentas, dados e prompts (o nosso servidor de fontes, o de edição, o glossário do exercício).

JSON-RPC 2.0: o formato das mensagens: um pedido com nome (`method`) e dados (`params`), e uma resposta com resultado (`result`) ou erro.

Transportes (como as mensagens viajam):

- **stdio:** o cliente abre o servidor como um programa no mesmo computador e conversa pela entrada e saída de texto. **Cuidado:** o servidor não pode imprimir nada na saída (`console.log`), porque é por ali que o protocolo conversa.
 - **Streamable HTTP:** para servidores na internet, pela web.
-

Slide 11 — O que um servidor oferece

Três tipos de coisa, e a diferença é **quem decide usar**:

- **Tools** (ferramentas): o **modelo** decide chamar no meio do trabalho. Ex.: `ler_fonte`.
- **Resources** (recursos): dados que a **aplicação** decide ler e colocar no contexto. Ex.: `noticia://n01`. Funcionam como endereços (URIs).
- **Prompts:** mensagens prontas que o **usuário** escolhe usar, como um modelo de pedido. Ex.: `classificar_noticia`.

Anotações (dicas sobre a ferramenta): "só lê" (`readOnlyHint`), "apaga ou muda algo" (`destructiveHint`), "repetir não muda o resultado" (`idempotentHint`). São **dicas**: a especificação manda não confiar nelas se o servidor não for confiável.

Slide 12 — Ao vivo: o protocolo

Rode `demo:protocolo` (instantâneo). Ela mostra as mensagens de verdade:

1. **tools/list:** "que ferramentas você tem?" O servidor responde com o nome e o formato de cada uma.
2. **tools/call:** "use esta ferramenta".
3. **Erro de ferramenta:** buscar um termo que não existe **não quebra nada**: volta uma resposta normal marcada com `isError`. Assim o modelo lê a mensagem e se corrige.
4. **resources/read** e **prompts/get**.

As duas "eras" do protocolo (explique simples):

- Na versão anterior (2025), a conversa começava com um "aperto de mão" (`initialize`) que abria uma sessão.

- Na versão atual (julho de 2026), **não há sessão**: cada pedido leva consigo a versão do protocolo. Isso simplifica servidores na nuvem.
- A demo mostra as duas. Para o que fazemos no curso, o resultado é o mesmo.

Pergunta provável: *"Por que o padrão do SDK ainda é a versão antiga?"* Compatibilidade: a maioria dos servidores existentes ainda fala a versão antiga. O cliente pode sondar a nova (modo `auto`) e voltar para a antiga se precisar.

Slide 13 — Segurança

MCP dá ao modelo acesso a dados e à execução de ações. Cuidados:

- **Menor privilégio**: cada agente só recebe as ferramentas de que precisa. No projeto, o coletor só pode listar e ler fontes; não pode publicar.
 - **Validar o que vem do modelo**: o teste do projeto tenta ler a fonte `../etc/passwd` (um truque para ler arquivos do sistema). O servidor confere se a fonte existe **antes** de abrir qualquer arquivo e responde com erro.
 - **O que o servidor devolve é dado não confiável**: pode conter injeção de prompt (Aula 3).
 - **Ferramentas que mudam algo** devem pedir aprovação humana (Aula 7).
 - **Só instale servidores de fontes confiáveis**: um servidor MCP é código rodando no seu computador.
-

Slide 14 — Divisória "4 · O time do curador"

Rode `demo:time` (versão curta, 3 itens, ~2 min): cada papel imprime o que está fazendo, e no fim aparece a edição em markdown gravada pelo servidor de edição.

Slide 15 — Resultado do time

Os números da execução completa (`m05:edicao`) estão nas **notas do mentor** do README do módulo. Comente:

- quanto tempo cada papel gastou (o classificador costuma ser o mais caro: uma chamada por notícia);
- se o coletor deixou notícias boas para trás (o mesmo risco do agente da Aula 4);
- se o revisor reprovou algo e se a segunda tentativa resolveu.

Leve a pergunta: este problema precisava de um time? Volte ao slide 8.

Slide 16 — Divisória "Discussão"

15 minutos.

Slide 17 — Para conversar (com respostas esperadas)

1. **"Precisava de quatro agentes?"** Honestamente, não muito: as etapas são em sequência e dependem umas das outras. O ganho real foi **organização**: cada papel tem uma tarefa só, ferramentas e permissões diferentes (o coletor só lê; só o publicador escreve) e pode ser testado e trocado separadamente. Um pipeline em código com as mesmas funções faria o mesmo.
2. **"Que padrão lembra um processo do seu time?"** Deixe as pessoas darem exemplos. Revisão de código (gerador + revisor), atendimento por níveis (handoff), aprovação de despesas (pipeline).
3. **"Tool x resource com a mesma informação?"** Muda quem decide buscar: a tool, o modelo, no meio do raciocínio; o resource, a aplicação, antes de chamar o modelo. Ter os dois dá flexibilidade.
4. **"Que servidor MCP você queria ter?"** Exemplos: consulta à documentação interna (só leitura), abrir chamados (muda algo, então pede aprovação), consultar métricas (só leitura).
5. **"Servidor de fonte desconhecida?"** É código rodando com o seu acesso: pode ler arquivos, vazsar dados, mentir nas descrições das ferramentas ou devolver textos com injeção.

Se o tempo apertar: 1, 3 e 5.

Slide 18 — Para a próxima aula

- **Exercício:** escrever um servidor MCP (o glossário de IA da mentoria) e os padrões multiagente como funções. O desafio implementa o handoff, com detecção de "ping-pong" (agentes passando a conversa um para o outro sem fim).
 - **Projeto final, etapa M5:** os dois servidores MCP, o adaptador e o time.
-

Slide 19 — Próxima aula

Gancho: **LangGraph** organiza o fluxo como um **grafo** (caixinhas ligadas por setas), com o estado salvo a cada passo (para retomar de onde parou) e caminhos que dependem de condições (por exemplo, da confiança).

Glossário rápido

Termo	Em uma frase
Multiagente	vários agentes, cada um com um papel, trabalhando na mesma tarefa
Pipeline	cada um faz uma etapa e passa adiante
Supervisor	um agente central escolhe quem trabalha a cada rodada
Hierárquico	supervisores de supervisores
Handoff	passar a conversa para outro agente
Gerador + revisor	um faz, outro critica, até aprovar
MCP	Model Context Protocol: padrão para plugar ferramentas e dados em aplicações com IA
Protocolo	regras para dois programas conversarem
Host / cliente / servidor	a aplicação / o conector / quem oferece as ferramentas
JSON-RPC	formato de mensagem: pedido com nome e dados, resposta com resultado ou erro
stdio	conversar com um programa local pela entrada e saída de texto
Streamable HTTP	conversar com um servidor pela web
Tool / resource / prompt	ferramenta (o modelo usa) / dado (a aplicação lê) / pedido pronto (o usuário escolhe)
URI	endereço de um recurso, como <code>noticia://n01</code>
Anotações	dicas sobre o que a ferramenta faz (só lê, apaga, pode repetir)
Menor privilégio	dar a cada parte só o acesso de que ela precisa
MCP Inspector	ferramenta para testar servidores MCP pelo navegador
SDK	biblioteca oficial para programar com uma tecnologia